

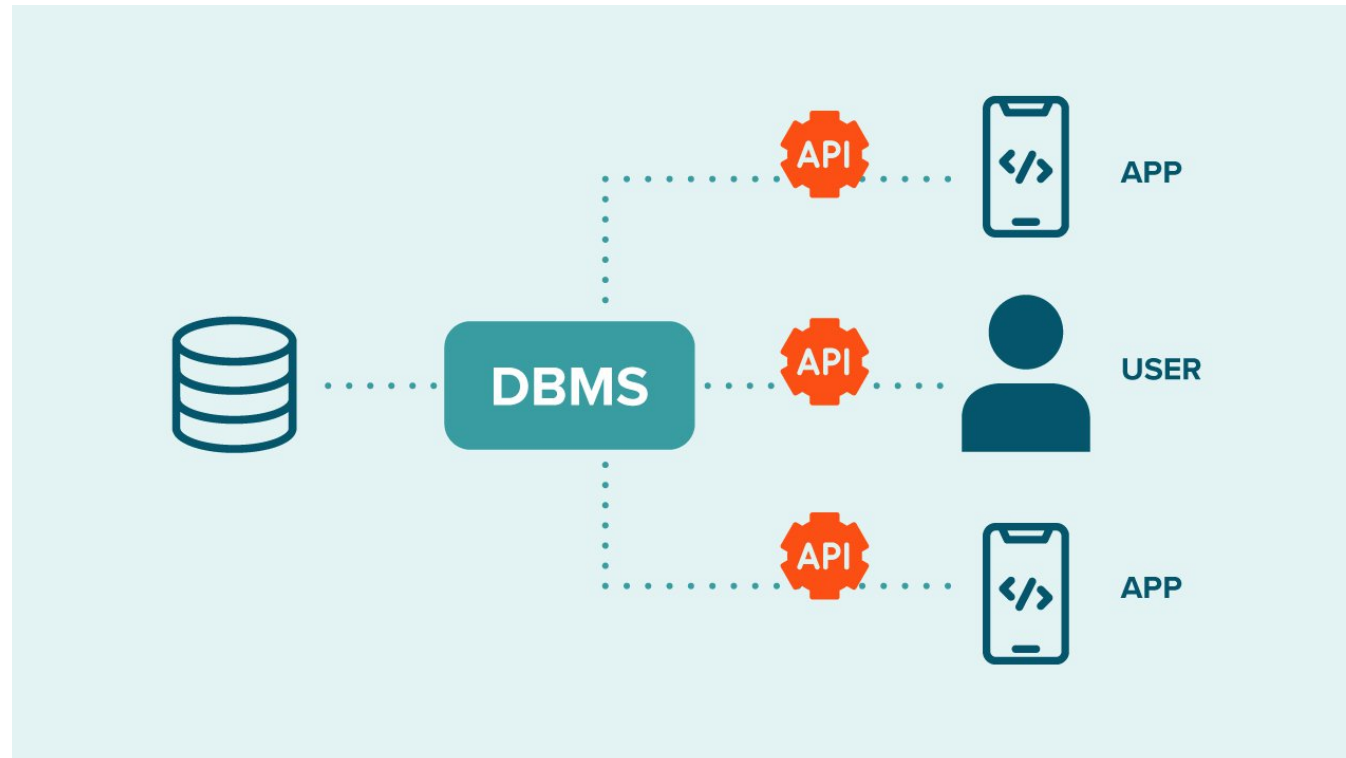
Specific objectives	Contents	Hours
Explain the basic <u>database</u> and database management system and data models	Unit I: Introduction 1.1 Introduction to Database Systems 1.2 Database Management System 1.3 Data models 1.4 Relational Model and Entity-Relationship Diagrams	4 hrs
Understanding the basic data modeling and fundamentals of SQL and queries.	Unit II: Data Modeling and SQL 1.1 Concept of Relations 1.2 Referential integrity and foreign key 1.3 Relational algebra 1.4 Various types of joins 1.5 SQL Fundamentals: DDL, DML 1.6 Advanced SQL (Joins, Subqueries, Aggregation)	4 hrs

Er. Yuba Raj Devkota
NCIT

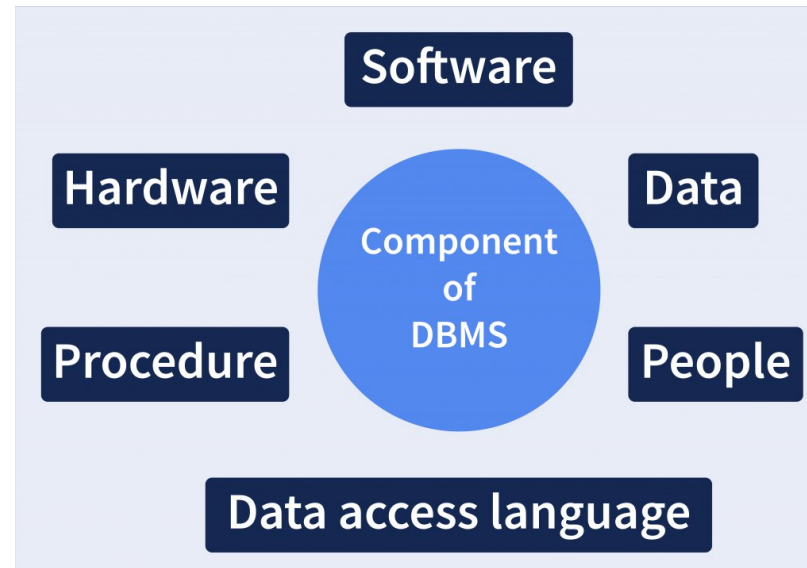
Introduction to Database System

Definition

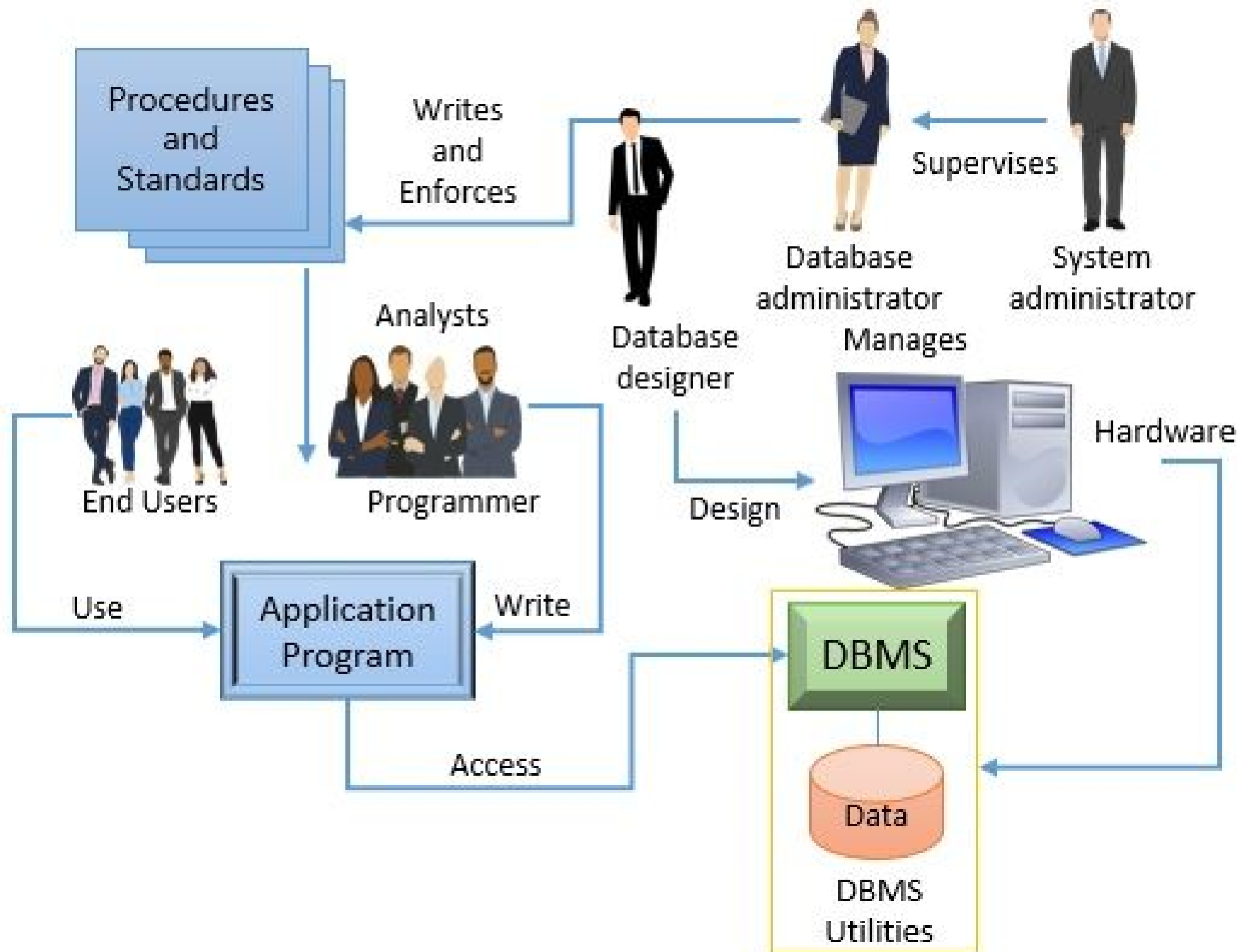
- A **Database System (DBS)** is a collection of interrelated data and a set of programs to access, manage, and update the data efficiently.
- It provides an organized way to store, retrieve, and manage data.



Components of a Database System



1. **Data** – Raw facts stored in tables (e.g., student records, employee details).
2. **Hardware** – Physical devices (servers, storage, I/O devices).
3. **Software** – DBMS software (e.g., MySQL, Oracle, PostgreSQL).
4. **Users** –
 - **End Users** (interact via applications)
 - **Database Administrators (DBA)** (manage & maintain DB)
 - **Application Developers** (create DB applications)



Advantages of DBMS

- **Redundancy Control** – Minimizes duplicate data.
- **Data Integrity & Consistency** – Ensures accuracy and validity.
- **Data Sharing** – Multiple users can access data simultaneously.
- **Security & Privacy** – Access control mechanisms.
- **Backup & Recovery** – Prevents data loss.

Disadvantages of DBMS

- **Complexity** – Requires expertise to design and manage.
- **Cost** – High initial setup and maintenance cost.
- **Performance Issues** – Large databases may slow down.

Database Management System (DBMS)

Definition

- A **DBMS** is software that interacts with users, applications, and the database to capture and analyze data.
- Examples: MySQL, Oracle, SQL Server, MongoDB.

Functions of DBMS

1. **Data Definition** – Defines structure (tables, constraints).
2. **Data Manipulation** – Insert, update, delete, query data.
3. **Data Security & Authorization** – Controls user access.
4. **Data Backup & Recovery** – Prevents data loss.
5. **Concurrency Control** – Manages simultaneous access.

What are the key differences between Structured, Semi Structured and Unstructured data [PU, 2025]

TYPE	STRUCTURE	FLEXIBILITY	EXAMPLE
Structured	Fixed, tabular	Low	SQL database table
Semi-Structured	Partial, tagged	Medium	JSON, XML, email headers
Unstructured	None, free-form	High	Photos, videos, WhatsApp messages

DATA TYPE	BEST DATABASE MODEL	EXAMPLE SYSTEMS
Structured	Relational (SQL)	MySQL, PostgreSQL, Oracle
Semi-Structured	Document / Key-Value NoSQL	MongoDB, Redis, Couchbase
Unstructured	Data Lakes / Search Engines	Amazon S3, Elasticsearch, Hadoop

Types of DBMS / DBMS Models

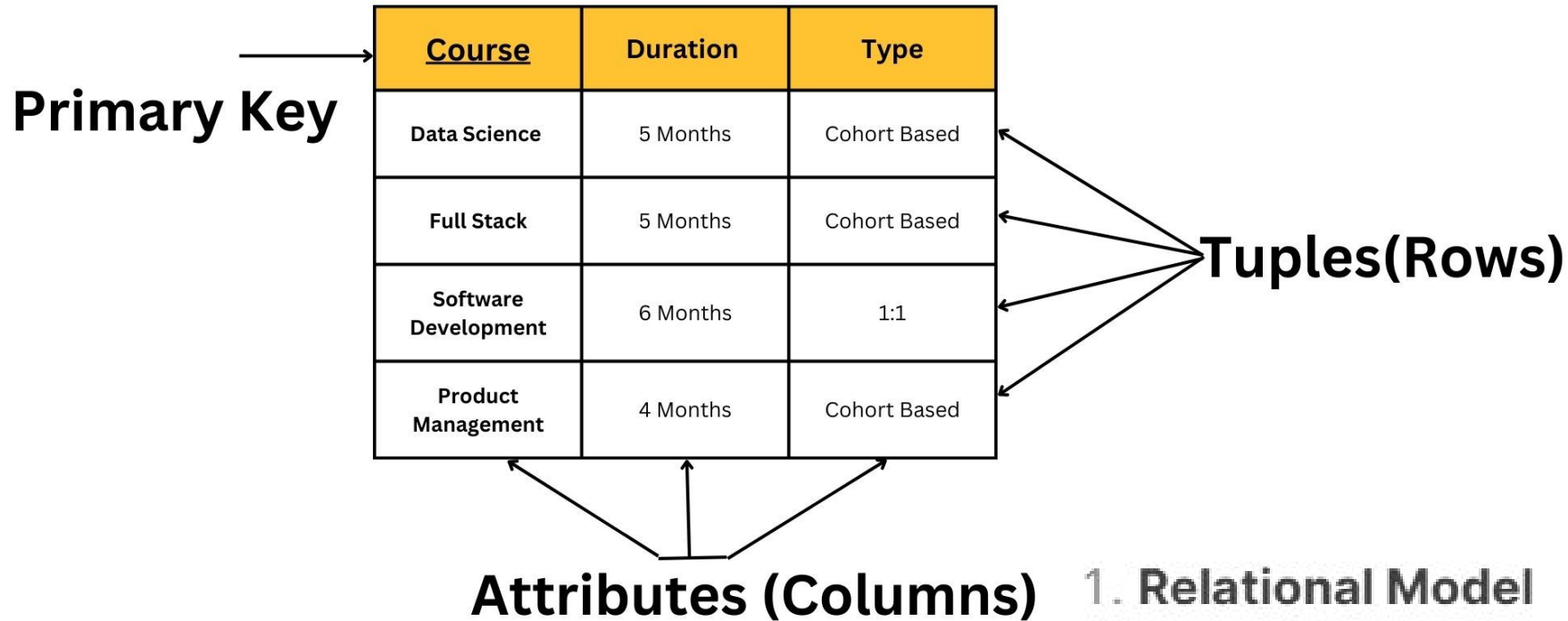
1. **Relational DBMS (RDBMS)** – Uses tables (e.g., MySQL, PostgreSQL).
2. **NoSQL DBMS** – Handles unstructured data (e.g., MongoDB, Cassandra).
3. **Hierarchical DBMS** – Tree-like structure (e.g., IBM IMS).
4. **Network DBMS** – Graph-like structure (e.g., IDMS).

- A **Data Model** defines how data is structured, stored, and manipulated in a database.

Comparison of Data Models

Model	Structure	Use Case
Relational	Tables (Rows/Cols)	Structured data (Banking, ERP)
NoSQL	Flexible (JSON, KV)	Big Data, Real-time apps
ER Model	Entities & Relations	Database Design
Object-Oriented	Objects & Classes	Complex data (CAD, GIS)

Relational Model in DBMS

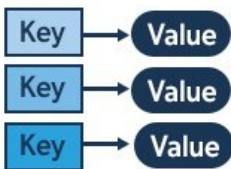


1. Relational Model

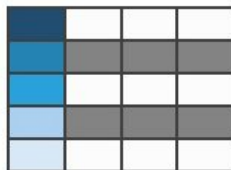
- Data is stored in tables (rows & columns).
- Uses **SQL** for queries.
- Example: MySQL, Oracle.

NoSQL

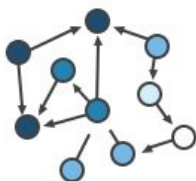
Key-Value



Column-Family



Graph



Document



- NoSQL is **best for unstructured, high-speed, and scalable applications** (social media, IoT, real-time analytics).
- **Twitter (X) uses MongoDB for tweets & Redis for caching**, proving NoSQL’s efficiency in real-world scenarios.

Types of NoSQL Databases

Type	Description	Example Use Case	Popular Databases
Document	Stores JSON-like documents	User profiles, CMS	MongoDB, CouchDB
Key-Value	Simple key-value pairs	Caching, session storage	Redis, DynamoDB
Column-Family	Stores data in columns	Big data analytics	Cassandra, HBase
Graph	Nodes & edges for relationships	Social networks, fraud detection	Neo4j, ArangoDB

Why Twitter Uses NoSQL?

Twitter (now X) handles **millions of tweets per second**, requiring:

- **High-speed read/write operations** (NoSQL is faster than SQL for this).
- **Scalability** (handles sudden traffic spikes).
- **Flexible schema** (tweets, likes, retweets have varying structures).

How Twitter Uses NoSQL (MongoDB & Redis)

1. Tweets & User Data (MongoDB – Document Store)

- Stores tweets as **JSON documents** (flexible schema).
- Example document:

json

```
{
  "tweet_id": "12345",
  "user": "@elonmusk",
  "content": "Hello Twitter!",
  "likes": 500000,
  "retweets": 20000,
  "hashtags": ["hello", "twitter"]
}
```

2. Real-time Analytics & Caching (Redis – Key-Value Store)

- Stores **trending hashtags, user sessions, and counters**.
- Example:

bash

```
SET "trending:#bitcoin" "500K tweets"
INCR "user:123:likes" # Increments like count in real-time
```

Explain why graph database is better suited for social network applications over relational database? [PU, 2025]

1. Natural Modeling of Relationships

Social networks are built on connections:

- Users → Friends
- Users → Posts → Likes/Comments
- Posts → Tags → Other Users
- Groups → Members

“ Graph: “Who is friends with whom?” → Direct edge traversal.

 Relational: “Join users → friendships → users” → Multiple JOINS, slow as data grows. ”

2. Efficient Traversal of Deep Connections

Graph databases excel at **multi-hop queries** — e.g., “Find all friends of friends who like hiking.”

This is called a **graph traversal**, and it’s blazing fast in graph DBs because relationships are physically linked in storage.

In relational databases, you’d need recursive CTEs or multiple self-JOINS — which become exponentially slower and harder to maintain as depth increases.

3. Flexible Schema for Evolving Features

Social apps constantly add new types of relationships:

- “Follows”, “Blocks”, “Mentions”, “Shares”, “Joins Group”

Graph databases handle schema evolution gracefully — you can add new node types or relationship types without altering existing structures.

Relational databases require schema migrations (ALTER TABLE) and complex redesigns when adding new relationship types.

4. Intuitive Query Language

Graph query languages like **Cypher (Neo4j)** or **Gremlin** let you write queries that *look like the graph itself*.

Example:

```
1 MATCH (user:User {name: "Alice"})-[:FRIEND]->(friend)-[:FRIEND]->(fof)
2 WHERE NOT (user)-[:FRIEND]->(fof)
3 RETURN fof.name
```

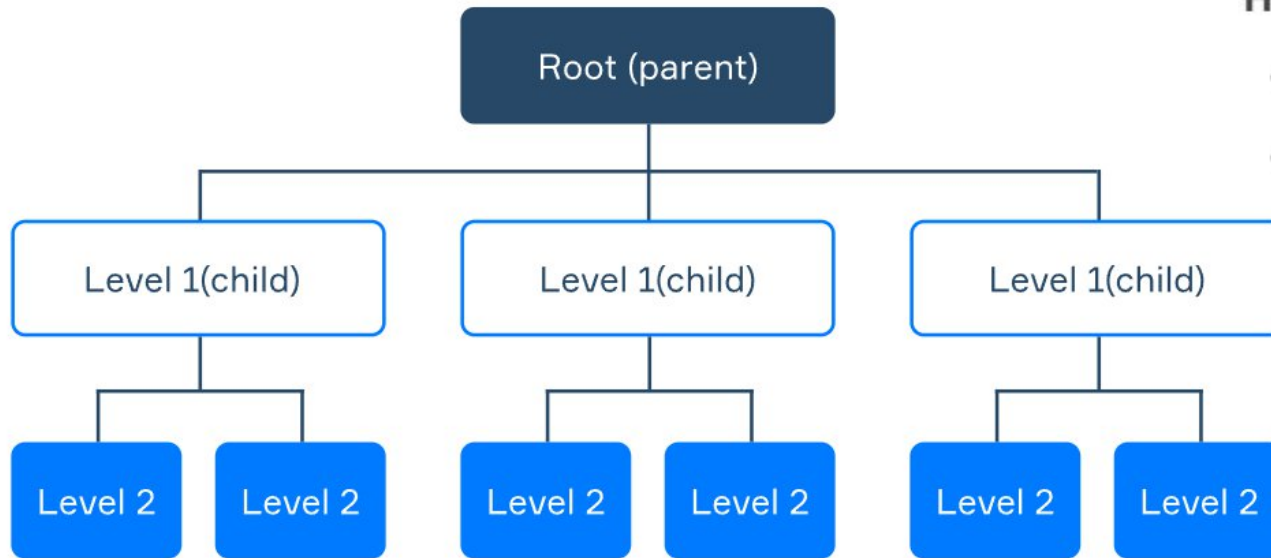
→ Finds “friends of friends” not already directly connected.

In SQL, this would be a nightmare of nested joins or recursive queries.

Real-World Use Cases

- **LinkedIn:** Uses graph tech for “People You May Know” and “How You’re Connected”.
- **Facebook:** Originally used MySQL but migrated parts to graph-based systems for friend recommendations.
- **Twitter:** Uses graph models for follow/follower networks and trending topics.

The Hierarchical Database Model



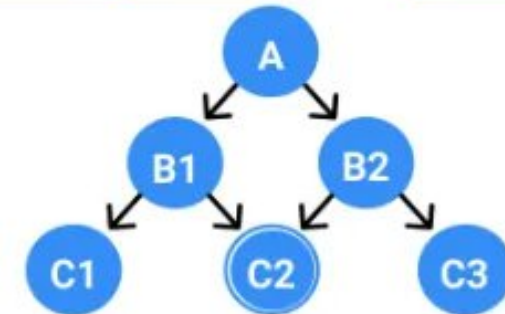
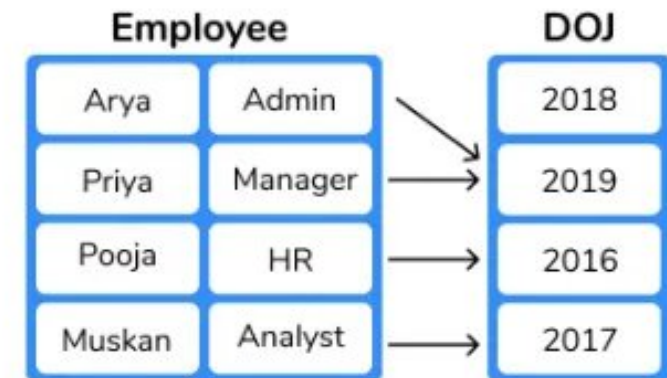
Advantages of DBMS over File Systems

- Eliminates data redundancy.
- Provides efficient query processing.
- Ensures ACID (Atomicity, Consistency, Isolation, Durability) properties.

Hierarchical & Network Models (Legacy Models)

- **Hierarchical** – Parent-child tree structure.
- **Network** – More flexible than hierarchical (many-to-many).

Network Model



Entity Relationship Model

An **Entity-Relationship (ER) Diagram** is a **visual representation** of the logical structure of a database. It illustrates:

- **Entities** (real-world objects like students, courses)
- **Attributes** (properties of entities like name, age)
- **Relationships** (how entities interact, e.g., "student enrolls in course")

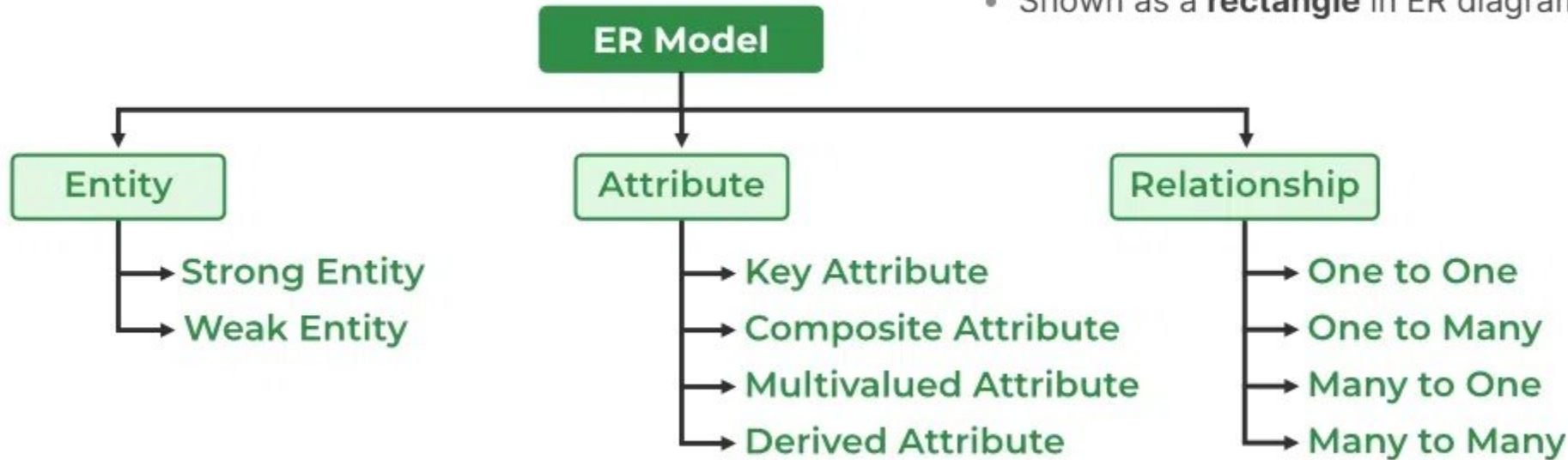
Purpose of ER Diagrams

- ✓ Helps in **database design** before implementation.
- ✓ Acts as a **blueprint** for developers and DBAs.
- ✓ Ensures **data integrity** by defining relationships clearly.

Components of ER Diagram

(A) Entities

- Represents a **real-world object** (e.g., Student, Employee, Product).
- Shown as a **rectangle** in ER diagrams.

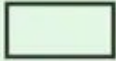
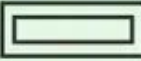


(B) Attributes

- **Properties** of an entity (e.g., student_id, name, age).
- Types:
 - **Simple vs. Composite** (e.g., "name" vs. "address" with sub-attributes).
 - **Single-valued vs. Multi-valued** (e.g., "age" vs. "phone_numbers").
 - **Derived** (e.g., "age" derived from "birth_date").
 - **Key Attribute** (Primary Key, e.g., "student_id").

Notation:

- **Oval shape** for attributes.
- **Underlined** = Key attribute.
- **Double oval** = Multi-valued.
- **Dashed oval** = Derived.

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

(C) Relationships

- Defines **how two entities interact**.
- Shown as a **diamond shape**.

2. One-to-Many (1:N)

- One entity relates to **many** others.
- Example: **Department** $\leftrightarrow$ **Employees** (one department has many employees).

3. Many-to-Many (M:N)

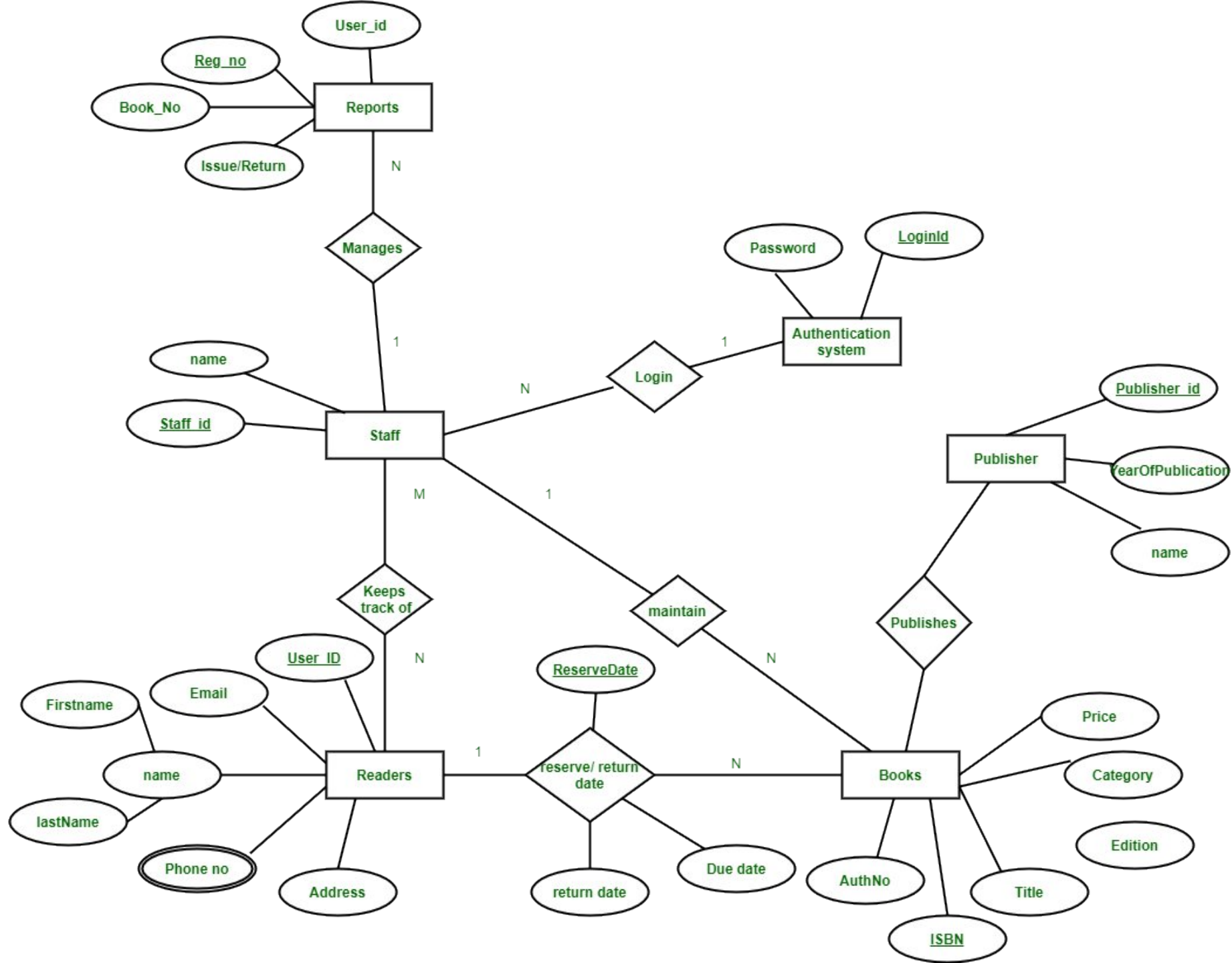
- Multiple entities relate to multiple others.
- Example: **Students** $\leftrightarrow$ **Courses** (many students enroll in many courses).

Types of Relationships:

1. One-to-One (1:1)

- One entity relates to **exactly one** other entity.
- Example: **Employee** $\leftrightarrow$ **Company Car** (one employee gets one car).

ER Diagram for the scenario of Library Management System



Concept of Relations

In relational databases, a **relation** is a mathematical term for what we commonly call a **table**. It consists of:

- **Rows (Tuples):** Represent individual records.
- **Columns (Attributes):** Define the fields/characteristics of the data.

Example: Student Relation

student_id (PK)	name	age	department
101	Alice	22	CS
102	Bob	23	EE
103	Charlie	21	CS

- **Relation Name:** Student
- **Attributes:** student_id, name, age, department
- **Primary Key (PK):** student_id (uniquely identifies each row).

Key Properties of Relations

1. **No Duplicate Rows:** Each row must be unique (enforced by primary keys).
2. **Unordered Rows:** The order of rows doesn't matter.
3. **Atomic Values:** Each cell contains a single, indivisible value (1NF).
4. **Unique Column Names:** No two columns can have the same name.

Referential Integrity and Foreign Keys

Referential Integrity

A constraint that ensures **relationships between tables remain consistent**. It guarantees that:

- A **foreign key (FK)** must reference an existing **primary key (PK)** in another table (or be `NULL`).
- Prevents "orphaned records" (e.g., a course enrollment without a valid student).

Example: Enrollments Table

enrollment_id (PK)	student_id (FK)	course_id (FK)	grade
1	101	CMP553	A
2	102	CMP553	B
3	103	CMP553	C

Foreign Key (FK)

A column (or set of columns) in one table that references the **primary key** of another table.

- **Foreign Keys:**
 - `student_id` references `Student(student_id)`.
 - `course_id` references `Course(course_id)`.

How Referential Integrity Works

1. **Insert:** Cannot add an enrollment with `student_id = 999` (if `999` doesn't exist in `Student`).
2. **Update:** Cannot change `student_id` in `Enrollments` to a non-existent value.
3. **Delete:**
 - **Restrict:** Block deletion of a student if they have enrollments.
 - **Cascade:** Delete all enrollments if the student is deleted.
 - **Set NULL:** Set `student_id` to `NULL` in `Enrollments` if the student is deleted.

```
-- Create tables with referential integrity
CREATE TABLE Student (
  student_id INT PRIMARY KEY,
  name VARCHAR(50),
  age INT
);

CREATE TABLE Enrollments (
  enrollment_id INT PRIMARY KEY,
  student_id INT,
  course_id VARCHAR(10),
  grade CHAR(1),
  FOREIGN KEY (student_id) REFERENCES Student(student_id)
  ON DELETE CASCADE -- Deletes enrollments if student is deleted
);
```

Real-World Analogy

- **PK:** Like a **Social Security Number** (unique to each person).
- **FK:** Like a **library card number** on a borrowed book (points back to a specific person).
- **Referential Integrity:** Ensures no book is checked out to a non-existent cardholder.

Primary Key (PK)	Foreign Key (FK)
Uniquely identifies a row.	References a PK in another table.
Cannot be <code>NULL</code> .	Can be <code>NULL</code> (if allowed).
Only one PK per table.	Multiple FKs per table.

Explain how a foreign key can be used to enforce referential integrity constraints, including appropriate SQL Syntax. [PU, 2025]

A **foreign key** ensures that a value in one table (child table) must exist in another table (parent table).

This prevents invalid or orphan records.

Example Scenario

- **Department** table (parent)
- **Employee** table (child) — dept_id must match a department in Department table.

SQL Syntax

1. Create Parent Table

sql

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

2. Create Child Table with FOREIGN KEY

sql

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```

How Referential Integrity Is Enforced

- If you try to insert an employee with a **non-existing dept_id**, it will be rejected.

```
INSERT INTO Employee VALUES (1, 'John', 99);  
-- Error: 99 does not exist in Department table
```

-
- You **cannot delete** a department if employees are linked to it (unless cascading is used).

```
DELETE FROM Department WHERE dept_id = 1;  
-- Error: Cannot delete because employee is using dept_id = 1
```

Relational Algebra

Relational algebra is a **procedural query language** that operates on relations (tables) and returns relations as results. It forms the theoretical foundation for **SQL**.

Key Characteristics

- **Operators:** Unary (single relation) and binary (two relations).
- **Closure Property:** Operations always return a new relation.
- **Used in Query Optimization:** DBMSs convert SQL to relational algebra for execution planning.

Core Relational Algebra Operations

(1) Selection (σ)

- **Purpose:** Filters rows based on a condition.
- **Symbol:** $\sigma_{\text{condition}}(\text{Relation})$

```
-- SQL Equivalent: SELECT * FROM Student WHERE age > 20  
 $\sigma_{\text{age} > 20}(\text{Student})$ 
```

Input (Student):

student_id	name	age
101	Alice	22
102	Bob	19

Output:

student_id	name	age
-----	-----	-----
101	Alice	22

(2) Projection (π)

- **Purpose:** Selects specific columns (eliminates duplicates).
- **Symbol:** $\pi_{\text{attributes}}(\text{Relation})$

```
-- SQL Equivalent: SELECT name, age FROM Student  
 $\pi_{\text{name, age}}(\text{Student})$ 
```

Output:

name	age
Alice	22
Bob	19

(3) Union (U)

- **Purpose:** Combines rows from two relations (must be **union-compatible**).
- **Symbol:** Relation₁ U Relation₂

```
-- SQL Equivalent: SELECT * FROM CS_Students UNION SELECT * FROM EE_Students
CS_Students U EE_Students
```

Input (CS_Students):

student_id	name
101	Alice

Input (EE_Students):

student_id	name
-----	-----
102	Bob

Output:

student_id	name
-----	-----
101	Alice
102	Bob

(4) Set Difference (−)

- **Purpose:** Returns rows in Relation₁ **not** in Relation₂.
- **Symbol:** Relation₁ − Relation₂

```
-- SQL Equivalent: SELECT * FROM Students EXCEPT SELECT * FROM Graduates
Students − Graduates
```

(5) Cartesian Product (×)

- **Purpose:** Combines all rows from two relations (creates all possible pairs).
- **Symbol:** Relation₁ × Relation₂

```
-- SQL Equivalent: SELECT * FROM Students, Courses
Students × Courses
```

(6) Rename (ρ)

- **Purpose:** Renames a relation or attributes.
- **Symbol:** ρ_{NewName}(Relation)

```
-- SQL Equivalent: SELECT * FROM Students AS S
ρS(Students)
```

Relational Algebra

SQL Equivalent

$\sigma_{age > 20}(Student)$

```
SELECT * FROM Student WHERE age > 20
```

$\pi_{name, age}(Student)$

```
SELECT name, age FROM Student
```

$Student \bowtie Enrollments$

```
SELECT * FROM Student NATURAL JOIN Enrollments
```

Various Types of Joins

Joins combine data from two or more tables based on related columns. Essential for querying normalized databases.

(1) INNER JOIN

- Returns only matching rows from both tables.

```
SELECT Students.name, Enrollments.course_id  
FROM Students INNER JOIN Enrollments  
ON Students.student_id = Enrollments.student_id;
```

(2) LEFT (OUTER) JOIN

- Returns all rows from left table + matching rows from right table (NULLs if no match).

```
SELECT Students.name, Enrollments.course_id  
FROM Students LEFT JOIN Enrollments  
ON Students.student_id = Enrollments.student_id;
```

(3) RIGHT (OUTER) JOIN

- Returns all rows from right table + matching rows from left table.

```
SELECT Students.name, Enrollments.course_id  
FROM Students RIGHT JOIN Enrollments  
ON Students.student_id = Enrollments.student_id;
```

NATURAL JOIN

Automatically joins on columns with same names.

```
SELECT * FROM Students NATURAL JOIN Enrollments;
```

Differentiate between a Cartesian Product and a Join in a relational databases.

1. Cartesian Product

- Combines **every row** of Table A with **every row** of Table B.
- No condition is used.
- Usually produces a **very large** result set.

Example

Table A has 2 rows, Table B has 3 rows → result = $2 \times 3 = 6$ rows.

```
SELECT *  
FROM Students, Courses;    -- No condition
```

This will pair **every student with every course**, even if they are unrelated.

2. Join

- Combines rows **based on a matching condition**.
- Returns only **related** rows.

Example

```
SELECT s.name, c.course_name
FROM Students s
JOIN Courses c
ON s.course_id = c.course_id;
```

This returns only **students enrolled in their actual course**, not all combinations.

Feature	Cartesian Product	Join
Condition	No condition	Uses matching condition
Output Size	Very large ($A \times B$)	Only related rows
Use Case	Rare, theoretical	Common in queries

SQL Fundamentals: DDL, DML

Data Definition Language (DDL)

Commands that define database structure:

(1) CREATE

sql

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    age INT CHECK (age > 16)  
);
```

(2) ALTER

sql

```
ALTER TABLE Students ADD COLUMN email VARCHAR(100);  
ALTER TABLE Students DROP COLUMN age;
```

(3) DROP

sql

```
DROP TABLE Students;
```

(4) TRUNCATE

sql

```
TRUNCATE TABLE Students; -- Keeps structure, removes all data
```

Data Manipulation Language (DML)

Commands for data manipulation:

(1) INSERT

sql

```
INSERT INTO Students (student_id, name) VALUES (101, 'Alice');
```

(2) UPDATE

sql

```
UPDATE Students SET name = 'Alice Smith' WHERE student_id = 101;
```

(3) DELETE

sql

```
DELETE FROM Students WHERE student_id = 101;
```

(4) SELECT

sql

```
SELECT name, age FROM Students WHERE department = 'CS';
```

Advanced SQL (Joins, Subqueries, Aggregation)

(1) Advanced Joins

Example (3-Table Join):

```
SELECT Students.name, Courses.title, Enrollments.grade
FROM Students
JOIN Enrollments ON Students.student_id = Enrollments.student_id
JOIN Courses ON Enrollments.course_id = Courses.course_id;
```

(2) Subqueries

a) Scalar Subquery (Returns single value)

```
SELECT name FROM Students
WHERE age = (SELECT MAX(age) FROM Students);
```

b) Correlated Subquery

```
SELECT s.name FROM Students s
WHERE EXISTS (
    SELECT 1 FROM Enrollments e
    WHERE e.student_id = s.student_id AND e.grade = 'A'
);
```

(3) Aggregation

a) GROUP BY

```
SELECT department, COUNT(*) as student_count  
FROM Students  
GROUP BY department;
```

b) HAVING (Filter groups)

```
SELECT department, AVG(age) as avg_age  
FROM Students  
GROUP BY department  
HAVING AVG(age) > 20;
```

Homeworks [SOE, pre-board]

1. Design ER model for conference management system that manages authors, papers, reviewers, sessions, and scheduling. Also write SQL DDL to convert this ER model to equivalent relational schemas.
2. Given relational schemas

Employee(eid, emp_name, address, supervisor_id)

Department(did, dep_name)

Project(pid, prj_name, did)

Workson(eid, pid, hours)

Write relational algebraic expression and SQL for performing following operations.

- i. List the name of all employee for computer department along with their supervisor Name.
- ii. Find the total number of projects from each department along with department name.
- iii. Find the employee's name who do not work in any project.
- iv. Find the name of each project with more than 3 employees.